MATH 174 - Numerical Analysis

Dani Nguyen

Fall 2025

Contents

1 Error

1.1 Approximation

Let x be an exact number.
Let y be an approximation of x.

Def: The absolute error is $|x - y|$.
Def: The relative error is $\dfrac{|x-y|}{|x|}$,

1.2 Round-off Error

Usually a machine uses a finite amount of memory to store a number.
machine $\leftarrow \pi$
Suppose a machine can store some number of digits.

1. Allocate a digit for sign, positive or negative.
2. Allocate a number of digits for the mantissa.
3. Allocate a digit for sign of the exponent.
4. Allocate a number of digits for the exponent.

Under rounding in the case of a 4 digit mantissa, $\pi = 0.3142 \cdot 10^1$, this is the machine # representation of pi.

Def: Round-off error are all errors arising from the error between actual # and machine #.

Notation:

$$\begin{aligned} x &= \text{real } \# \\ fl(x) &= \text{machine } \# \text{ representation of } x \end{aligned}$$

In an s-digit rounding machine,
Abs error $= |x = fl(x)|$
Rel error $= \dfrac{|x - fl(x)|}{|x|} \leq \frac{1}{2} \cdot 10^{1-5} = 5 \cdot 10^{-5}$

1.3 Floating Point Arithmetic

What happens under arithmetic operations? (+, -, ·, /)

Two things to watch out for:

1. Many operations can degrade error.

2. Catastrophic cancellation.

1.3.1 Degrading Error

number	machine
x	$fl(x)$
y	$fl(y)$
$x + y$	$fl(fl(x) + fl(y))$

The more nested $fl()$ there are, the larger the error will be. The same follows for -, ·, /.

When performing arithmetic, it temporarily allows for more digits than the mantissa allows.

Algorithmic "solutions"
$((x + y) + z) + w$
Worst case, 4 times the error.
$(x + y) + (z + w)$
Worst case, 3 times the error.

1.3.2 Catastrophic Cancellation

Ex: 3-digit rounding machine. Given 2 numbers: 0.312 and 0.311.

$$0.312 - 0.311 = 0.001 = 0.1 \cdot 10^{-2}$$

Loss of significant digits!

Ex: 5-digit rounding machine. Given 2 numbers:

$$\begin{aligned}
x &= 0.41626324 \\
y &= 0.41626323 \\
x - y &= 0.00000001 \\
\text{In machine:} & \\
fl(x) &= 0.41626 \\
fl(y) &= 0.41626 \\
fl(x) - fl(y) &= 0.00000 = 0 \\
fl(fl(x) - fl(y)) &= 0 \\
\text{Relative error:} & \\
\frac{|x - y - fl(fl(x) - fl(y))|}{|x - y|} &= \frac{|x - y - 0|}{|x - y|} \\
&= 1 = 100\% \text{ error!}
\end{aligned}$$

2 Speed

2.1 Dot Product

Consider dot product of vectors

$$\vec{x} \in \mathbb{R}^n, \vec{y} \in \mathbb{R}^n$$

$$\begin{aligned}
\vec{x} \cdot \vec{y} &= \begin{bmatrix} x_1 & x_2 & \dots & x_n \end{bmatrix} \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} \\
&= x_1 y_1 + x_2 y_2 + \dots + x_n y_n
\end{aligned}$$

Count time-consuming operations:

1. Add/subtract
2. Multiply/divide
3. Comparisons
4. Memory access/storage

These are called FLOPS (floating point operations).

For the dot product, there are n multiplications and n-1 additions.
Adding these together, we get $2n - 1$ FLOPS.
Note that this expression is linear in n.

For large n, 2n dominates, $= \mathcal{O}(n) \approx$ constant times n.
ex: For n vs $2n$, $2n$ will take twice as long.
ex: For $\mathcal{O}(n^2)$, n vs $2n$, $2n$ will take four times as long.

2.1.1 MATLAB Dot Product Function

A simple MATLAB function to compute the dot product of two vectors and return the number of FLOPS used.

```
function [thedotproduct, theflops] = dotprod(x, y, n)
    thedotproduct = x(1)*y(1);
    theflops = 1;
    for i = 2:n
        thedotproduct = thedotproduct + x(i)*y(i);
        theflops = theflops + 2;
    end
end
```

2.2 Memory

Consider a matrix $A_{n\times n}$

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \dots & a_{nn} \end{bmatrix} vs \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

The square matrix uses n^2 memory locations, while the vector uses n memory locations. Each number also uses 8 bytes, creating another goal for algorithm development, <u>reduce memory usage</u>.

3 Gaussian Elimination

3.1 Triangular Matrices

Write a MATLAB function that when given $A\vec{x} = \vec{b}$, computes $\vec{x}$

$$A\vec{x} = \vec{b}$$

$$\text{A is triangular and invertible}$$

Let's say A is lower triangular. $a_{ij} = 0, \forall j > i$

$$\begin{bmatrix} a_{11} & 0 & 0 & \dots & 0 \\ a_{21} & a_{22} & 0 & \dots & 0 \\ a_{31} & a_{32} & a_{33} & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & a_{n3} & \dots & a_{nn} \end{bmatrix}$$

Similarly, if A is upper triangular, $a_{ij} = 0, \forall j < i$.
Ex: $A\vec{x} = \vec{b}$, A is upper triangular.

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a_{22} & a_{23} \\ 0 & 0 & a_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

This gives us the following system of equations:

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 &= b_1 \\ a_{22}x_2 + a_{23}x_3 &= b_2 \\ a_{33}x_3 &= b_3 \end{aligned}$$

So $x_3 = \frac{b_3}{a_{33}}, a_{33} \neq 0$ if A is invertible.

Then, $x_2 = \frac{b_2 - a_{23}x_3}{a_{22}}, a_{22} \neq 0$ if A is invertible.

Finally, $x_1 = \frac{b_1 - a_{12}x_2 - a_{13}x_3}{a_{11}}, a_{11} \neq 0$ if A is invertible.

This is called back substitution.
For $i = n, n-1, \ldots, 1$

$$
\begin{aligned}
x_i &= \frac{b_i - (a_{i,i+1}x_{i+1} + \ldots + a_{i,n}x_n)}{a_{ii}} \\
&= \frac{b_i - \sum_{j=i+1}^{n} a_{ij}x_j}{a_{ii}}
\end{aligned}
$$

Ex: $A\vec{x} = \vec{b}$, A is lower triangular.

$$
\begin{bmatrix} a_{11} & 0 & 0 \\ a_{21} & a_{22} & 0 \\ a_{31} & a_{32} & a_{33} \end{bmatrix}
\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} =
\begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}
$$

This gives us the following system of equations:

$$
\begin{aligned}
a_{11}x_1 &= b_1 \\
a_{21}x_1 + a_{22}x_2 &= b_2 \\
a_{31}x_1 + a_{32}x_2 + a_{33}x_3 &= b_3
\end{aligned}
$$

This is called forward substitution.
For $i = 1, 2, \ldots, n$

$$
\begin{aligned}
x_i &= \frac{b_i - (a_{i,1}x_1 + \ldots + a_{i,i-1}x_{i-1})}{a_{ii}} \\
&= \frac{b_i - \displaystyle\sum_{j=1}^{i-1} a_{ij}x_j}{a_{ii}}
\end{aligned}
$$

Now given a regular matrix, make it triangular without changing the solutions. Recall Gaussian elimination.

1. E_i (equation i) replaced by cE_i. Row scaling.

2. E_i swapped with E_j. Row swapping.

3. E_i replaced by $E_i + cE_j$. Row addition.

3.1.1 Speed of Back Substitution

For $i = n, n-1, \ldots, 1$

$$x_i = \frac{b_i - \sum_{j=i+1}^{n} u_{ij} x_j}{u_{ii}}$$

$$\begin{array}{ll} i = n & 1 \text{ flop} \\ \quad n-1 & 3 \text{ flops} \\ \quad \vdots & \vdots \\ \quad 1 & 2n-1 \text{ flops} \end{array}$$

$$\sum_{j=1}^{n}(2j-1) = 2\sum_{j=1}^{n}(j) - 1 = 2\left(\frac{n(n+1)}{2}\right) - 1 = n^2 + n - 1$$

With a large n, the term n^2 dominates, meaning $\mathcal{O}(n^2)$

3.2 Augmented Matrices

Gaussian Elimination on same matrices and different RHS comes up often in applications and algorithms.

3.2.1 LU Factorization

$$A = LU$$

where L is lower triangular with 1's in the diagonal and U is upper triangular

The important entries of L are called multipliers. These entries can be found with the negative of the coefficients of the steps of Gaussian elimination.

$$\begin{array}{rl} L\vec{y} = \vec{b} & \text{get y from forward substitution} \\ U\vec{x} = \vec{y} & \text{get x from back substitution} \end{array}$$